

E-Gaze: Gaze Estimation With Event Camera

Nealson Li ^{id}, *Student Member, IEEE*, Muya Chang ^{id}, *Member, IEEE*, and Arijit Raychowdhury ^{id}, *Fellow, IEEE*

Abstract—Near-eye gaze estimation is a task that maps the recording of an eye captured by an adjacent camera to the direction of a person's gaze in space. In contrast to frame-based cameras, event cameras are characterized by high sensing rates, low latency, sparse asynchronous data outputs, and high dynamic range, which are well suited for recording the fast eye movements. However, algorithms and system designs that operate on frame-based cameras are not applicable to event-based data, due to the natural differences in the data characteristics. In this work, we study the pattern of near-eye event-based data streams and extract eye features to estimate gaze. First, by analyzing eye parts and movements, and harnessing the polar, spatial, and temporal distribution of the events, we introduce a real-time pipeline to extract pupil features. Second, we present a recurrent neural network with a proposed coordinate-to-angle loss function to accurately estimate gaze from pupil feature sequence. We demonstrated that our system achieves accurate real-time estimation with angular accuracy of 0.46° and update rates of 950 Hz, thus opening up avenues for novel applications. To our knowledge, this is the first system that operates only on event-based data to perform gaze estimation.

Index Terms—Event camera, extended reality (XR), eye tracking, feature extraction, gaze estimation, recurrent neural networks.

I. INTRODUCTION

XR INCLUDES AR (augmented reality), VR (virtual reality) and MR (mixed reality) [1]. The applications span from healthcare to gaming [2], [3], [4], [5], [6], [7], [8], and XR headsets are emerging as potential next-generation mobile devices [9]. However, achieving a smaller form factor with an enhanced user experience is crucial for widespread adoption. Gaze estimation is one of the challenging tasks among active XR research areas. Gaze information is essential for implementing foveated rendering, an optimization technique capable of reducing image rendering workload by 80% [10], all while preserving the visual experience. Pupil steering, which shifts the viewpoint to the pupil location [11], is indispensable for Maxwellian-view display [12] in AR. Distortion correction [13] and varifocal display [14] also rely on precise gaze estimation. Additionally,

gaze serves as a controller-free approach to interact with XR user interfaces [15]. In extended reality, gaze plays a pivotal role in user perception and serves as a mode of interaction [16].

Gaze direction indicates where an individual is looking and their attention. To determine an individual's gaze, it is necessary to track the orientation of their eyes within their face, typically through visual information sources [17], [18], [19], [20]. A camera integrated into the headset's frame is positioned inward, directed towards the eye to capture eye movements. This setup falls under the category of near-eye tracking [21], [22], as the camera is situated close to the eye, with the eye occupying most of the camera's field of view. Using eye movements as input data, the task is to estimate the direction of a person's gaze in three-dimensional space as the output.

The eye exhibits two of the fastest movements in the human body, with blinks lasting a mere 100ms and saccades reaching speeds of $700^\circ/\text{s}$ [22], [23]. Additionally, there exist various involuntary eye movements, and their size can vary among individuals. Designing a system capable of tracking the eye under diverse lighting conditions and accurately estimating gaze is a challenging research topic. Moreover, the targeted platform for this work is the XR headset. Consequently, strict power constraints are imposed on this mobile device, along with demanding low-latency requirements for real-time operation.

The combination of a frame camera (RGB camera) and neural network has become a popular approach for gaze estimation [21], [24]. The frame camera captures the entire frame at a fixed rate, e.g., 1080×720 pixels per frame at a 60-frames-per-second rate. However, this data format is spatially dense, containing excess data that adds unnecessary workload, and it has low temporal resolution, leading to a loss of fine temporal detail when tracking rapid eye movements. While these systems can leverage cumulative advancements in frame-based computer vision research [25], achieving the desired specifications for XR applications remains a challenge. Additionally, such systems often require the integration of an IR LED (infrared light emitting diode) [26], [27], [28], adding an extra hardware component to the setup.

A novel alternative to the frame camera is the event camera [29]. The event camera senses the brightness changes of individual pixels. When a pixel's brightness change exceeds a certain threshold, either positively or negatively, the event camera records the coordinate, polarity, and timestamp as an event in the format (x, y, p, t) . Multiple events are recorded to capture rapid movements at microsecond resolution, and no events are recorded when there is no motion, except for noise. The event camera's characteristics, including low latency, high temporal resolution, high dynamic range, asynchronous operation, and

Manuscript received 7 February 2023; revised 10 October 2023; accepted 21 January 2024. Date of publication 29 January 2024; date of current version 5 June 2024. This work was supported in part by CoCoSys, one of seven centers in JUMP 2.0 and in part by a Semiconductor Research Corporation (SRC) program sponsored by DARPA. Recommended for acceptance by G. Gallego. (Corresponding author: Nealson Li.)

Nealson Li and Arijit Raychowdhury are with the School of Electrical and Computing Engineering, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: nealson@gatech.edu; arijit.raychowdhury@ece.gatech.edu).

Muya Chang is with NVIDIA, Santa Clara, CA 95051 USA (e-mail: mchang87@gatech.edu).

Digital Object Identifier 10.1109/TPAMI.2024.3359606

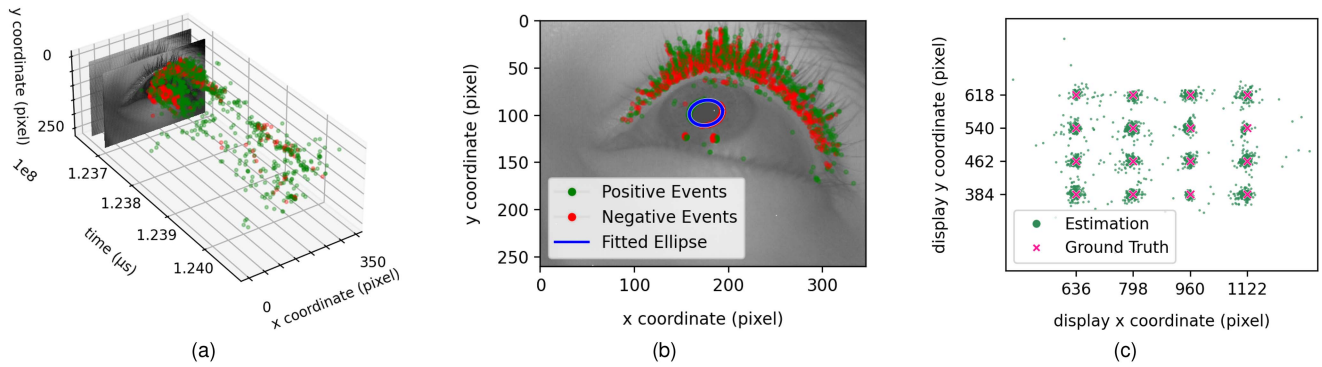


Fig. 1. (a) Eye movements captured as 2000 consecutive events (red and green dots) scattered spatially and temporally in 3D space (x, y, time). Two frames captured earlier in time are also displayed as reference to contrast the spatio-temporal difference between the two data formats. (b) Eye movements displayed without the time dimension. In the eye tracking stage, without using frame-based data, we extracted pupil features (blue ellipse) solely from event-based data. (c) System accuracy. This figure shows 16 gaze targets (pink cross) and all their associated estimations (green dots) displayed on the screen. Our system achieves sub-1° gaze estimation accuracy with an end-to-end latency of approximately 1ms.

sparse data acquisition, make it a better choice compared to the frame camera for gaze estimation.

Despite the great properties of event cameras, approaches for extracting information from event-based data are still being actively explored, particularly in their application to computer vision tasks. While there have been some efforts [28], [30] utilizing near-eye event cameras for eye tracking, none of them rely solely on event cameras for gaze estimation. One of these systems [28] requires the use of multiple IR LEDs to perform eye tracking based on event data streams and does not estimate gaze direction. The other work [30] relies on both IR LEDs to create glints in the eye and frame-based data, in addition to event-based data. The design of an event camera-exclusive gaze estimation system is crucial for fully harnessing the benefits of its sparse, asynchronous, and low-power properties.

This work sets several goals to ensure that the system can seamlessly integrate with XR headsets without complex setup. First, the system must meet the stringent low-latency requirements ($< 5ms$) [31] for XR applications while achieving high accuracy in gaze estimation ($< 0.5^\circ$) [14]. Furthermore, it should not necessitate additional hardware beyond the event camera, such as LED light sources or frame cameras. Thus, without IR LEDs, there will be no glints in the data stream to serve as features, and the system must rely solely on eye features. Due to the absence of a frame camera, the system must operate exclusively on event-based data without the assistance of frame-based data as a reference.

This paper presents a system that estimates gaze using event-based data stream, as shown in Fig. 1(a). The system extracts pupil features in the form of ellipses from the accumulated events, encoding eye movements as depicted in Fig. 1(b). Then, a sequence of these pupil features is input into a many-to-one recurrent neural network (RNN), followed by fully connected layers, to predict gaze direction as coordinate on the display, as shown in Fig. 1(c). This system successfully meets the constraints of XR applications and, to the best of our knowledge, is the first system that relies solely on event-based data for gaze estimation. The contributions of this work are as follows:

Contribution 1: We introduce a method for extracting eye features from event-based data, developed through keen observations and insights into the spatio-temporal variations in event density and polarity associated with different eye parts and their movements when captured using event cameras. This method combines image processing techniques and non-parametric statistical data analysis to efficiently extract pupil features within the target latency. The asynchronous nature of event-based data is preserved during the extraction process, resulting in asynchronous features. Importantly, this method is generalizable to different subjects with different eye part sizes, and is robust to blinking without a blink detector.

Contribution 2: In this study, we introduce a RNN architecture designed specifically for gaze estimation using event camera data. Recognizing the importance of temporal information in achieving highly accurate gaze estimation, we demonstrate that an architecture incorporating Long Short-Term Memory (LSTM) layers followed by dense layers can achieve an accuracy of $< 1^\circ$ with only 83,782 parameters. We have proposed an angular loss function because the L2 norm of the coordinate alone does not adequately account for the fact that a point on the display that is farther from the eye has a lesser impact on the actual angle error compared to a point that is closer. The custom loss function takes into account the distance of the point from the eye and serves as a more suitable target for the neural network to optimize.

Contribution 3: We demonstrate that, with a minimal hardware setup consisting only the event camera, the system is able to extract features from event-based data and estimate gaze on-demand with high accuracy and low latency. Our extensive experiments utilize an event-based near-eye dataset collected from 24 subjects, and this dataset includes a wide range of eye sizes, shapes, and positions. Our results shows the robustness of the near-eye event camera system in accommodating variations among different subjects. It achieves an accuracy of 0.46° with latency consistently below 1.025ms.

The rest of the paper is organized as follows. Section II outlines the related works in near-eye gaze estimation.

Section III formulates the gaze estimation problem and provides an overview of the proposed system. Section IV describes the event-based eye tracking approach, followed by the explanation of the gaze estimation RNN architecture in Section V. Next, Section VI details the event dataset in use and the experimental setup. Section VII reports the results including accuracy and latency. Finally, Section VIII outlines the limitations of the system, and Section IX concludes this paper.

II. RELATED WORK

Most near-eye gaze estimation systems are frame-based [26], [27], [32], [33], [34], [35], [36], [37], and only two works [28], [30] utilize event data in part of their systems. In general, the system architectures consist of two major building blocks regardless of the camera type, eye tracking and gaze estimation. Eye tracking block focuses on eye feature extraction from data stream, and gaze estimation block is responsible for mapping the eye features to gaze direction. In this section, eye tracking and gaze estimation approaches in frame-based systems are first elaborated. Due to the novelty of the field, introduction of the event camera is the next topic, followed by feature detection and tracking methods in the event stream. The two event-based near-eye systems are elaborated in the last paragraph of this section.

Frame-Based Eye Tracking: Eye tracking is a computer vision task that consists of two steps, segmenting the eye into eye parts and extracting each part as features. Implemented with traditional computer vision techniques, [32] segments the frame with image processing algorithms such as canny edge detection and blob detection, and performs ellipse fitting to extract pupil features from the blobs. With the rapid progress in deep learning, several works leverage the power of neural network to perform near-eye tracking. Specifically focused on segmentation, [33] proposed a convolutional neural network based on U-Net architecture to classify each pixel into one of four categories, background, sclera, iris, or pupil. The architecture implements boundary aware loss that utilizes information generated by canny edge detection and improves the completeness of each region. Built on the foundation [33] laid, [34] focuses on extracting eye features from segmentation results. The authors introduced a regression module that is an add-on to existing eye segmentation CNN, and the entire architecture can be trained end-to-end. The regression module inputs intermediate segmentation information and predicts the ellipse parameters for pupil and iris. The work also introduces center of mass loss and metrics such as goodness of fit to improve the result. To reduce the system workload, [35] proposed an eye tracking method that predicts the region of interest (ROI) with events generated by emulated event camera and performs segmentation with a U-Net-like architecture on frame-based data. The work shows that ROI prediction simplifies the network and reduces the data transmission from sensor. To predict eye activities such as blinking simultaneously, in [27], a multitask DNN with single feature encoder and six task decoder is proposed to extract 2D eye geometries and eye expressions from a frame. [36], [37] focused on real-world challenges [38] beyond lab setup, including

illumination variation, eyewear reflection and more. [36] proposed a pupil detection algorithm utilizes edge filtering and oriented histogram, while [37] developed an eye tracking system that applies morphological operation and ellipse selection, and both showed robustness against the non-ideal scenarios.

Frame-Based Gaze Estimation: Commonly by geometric approaches, gaze direction is estimated from the 2D eye features detected and tracked. A 3D eye model is reconstructed from the 2D eye features extracted in the previous eye tracking stage. In 3D space, the line that connects the center of pupil and the center of eye ball is the approximation of gaze direction. There are two methods to fit a 3D eye model, glint based and contour based. Glint based method requires IR LEDs additionally to generate reflections on eye ball that are modeled as specular reflections on a sphere. Contour based method is a glint-free method. The method analyzes the shape of the pupil with the eye staring at different directions, and estimates the 3D eye model. [32] and [35] incorporate the contour method, and fitted a eye with regression. [27] facilitates a hybrid method since both pupil contour and glints are required to estimate the 3D geometry of the eye. To account for the difference of optical axis(perpendicular to the cornea center) and visual axis(actual gaze direction), [27] also introduced a neural network to map optical axis to visual axis. Unlike other frame-based systems, [26], estimates gaze directly from input image without tracking eye features. The system architecture contains convolutional layer followed by dense layer and operates in high update rate. However, the dataset contains no noisy movement such as blink, and the camera is on-axis, i.e. right in front of the eye blocking the sight, instead of off-axis like common head-mounted displays (HMDs) are designed.

Event-Based Feature Detection and Tracking: The event camera, DAVIS [39] is a sensor that logs the events triggered when intensity difference exceeds a threshold. Due to its novelty and distinct data format, tracking feature with event-based data stream is an active research topic. In [40], an event-based Hough circle detection method that utilizes optical flow information is proposed to track ball in a cluttered event space. The detection rate is limited to 500Hz when operating offline on a controlled dataset with slow ball movement, and only half of the time the ball is detected. [41] tracks 4 indoor robots with event camera by using DBSCAN (Density-based spatial clustering of applications with noise) [42] to cluster events into robots. Various clustering algorithms are compared in terms of event data clustering ability and computational time. Although DBSCAN is able to reject noise and correctly detect robots within the least time, the 11ms computational time for clustering alone is still too costly for a latency critical platform like XR headsets. [43] experimented the adaptation of LSTM architecture for asynchronous event data stream. The work tracks a bouncing ball in a stationary event camera setup by extracting the mean position, and predicted the trajectory. Compared to sequence that is sampled in fixed rate, the paper showed that the LSTM architecture outputs lower error predictions with sequence sampled asynchronously and spatially.

Event-Based Eye Tracking and Gaze Estimation: The system [44] proposed tracks the bounding boxes of the subject's face and eye with event data stream that captures the upper

body. Although [44] tracks eye, it is not a near-eye setup, does not locate eye features, and does not estimate gaze. At the time of finishing this work, there are only two related near-eye event camera gaze estimation work, and both are not fully event-based. The first work, [30], takes frame-based data as input along with event-based data. The system relies on the feature extraction pipeline that operates on frame-based data to locate eye parts, and then combines the event data to update the eye features. It is still an approach that requires frame-based data and RGB camera to extract eye features. The work proposed a polynomial regressor to then map eye features to gaze direction. The second work, [28], implemented eye tracking only without gaze estimation, and is not a complete gaze estimation system. It operates solely on event camera to track eye without requiring frame-based data. However, the system relies on a set of 10 LEDs to generate glints as markers to perform corneal sphere regression. Additionally, while operating at high frequency, the coded differential lighting scheme on LEDs is constrained by the event camera sensor bias [45], which limits the system operation rate to 1000Hz.

III. FORMULATION

We introduce the event-based gaze estimation system and elaborate on the advantages of event camera over frame-based camera for such task in this section. Then, we formulate the event-based gaze estimation problem and describe the high level system architecture.

A. System Definition

Event-based near-eye gaze estimation system is a system that has the event camera mounted close to one of the eyes, and estimates the direction of gaze with the event data stream.

Event Camera: Events are generated while the brightness difference of an individual pixel crossed a threshold, and this operation provides event camera with unique properties. First, the event camera is motion-driven. The event camera logs only the changes generated by motion, and ignores redundant fixed information. Events are triggered at a higher rate when the motion is faster, and with no movement, no event except noise is generated. Therefore, the event data stream is asynchronous. Second, events are generated at the granularity of a single pixel, not the granularity of a frame, which contains all the pixels.

Near-Eye System: In a near-eye system setup, the camera is positioned close up to the face between the eye and cheek, and usually integrated into the headset. The camera points directly toward the eye with only the eye in the camera field of view (FoV). The camera configuration of this work is off-axis, thus the eye recorded is the frontal view of the eye with projective transformation. A near-eye system usually has cameras setup for both eyes, and has two data streams. This work focuses on the event data stream from only the cameras on the left side of all subjects.

Eye and Gaze: Gaze is the direction where the eye is looking at, and it is an invisible line in space. Therefore, we rely on observing and analyzing the visible subject, eye, to estimate the gaze. In the FOV of a near-eye camera, these eye parts

are visually present: eyelashes, eyelids(upper and lower), iris, and pupil. The relative position of each eye part is person independent, and the position of the entire eye in the FoV differs in each usage, since variations in the relative position of the eye and the camera is inevitable each time wearing the headset. On top of the variations in a still eye, there are motions. Eye movements includes fixation, saccade, blink and more subtle voluntary and involuntary movements.

B. Event Based Gaze Estimation Problem

The algorithm that inputs the near-eye event data stream and outputs gaze estimation is the focus of this work.

Event Data: The output of the event camera is a stream of events $\mathcal{E}$. An event is expressed as $e = (x_e, y_e, p_e, t_e)$. (x_e, y_e) is the pixel coordinate of the triggered event in the event camera field of view. p_e is polarity and indicates whether the threshold is crossed positively or negatively, in other words, the brightness increased or decreased. t_e is the timestamp when the event is triggered.

Gaze: Gaze is the direction of the eye stares at in space. In a head mounted display setup, the gaze direction is the vector pointing from the eye to the pixel on the display that the eye is looking at. It is expressed by the display pixel coordinate (x_d, y_d) , where the line of gaze direction intersects the display plane.

The problem this work solves is, given an event data stream of the eye $\mathcal{E} = \{e_i = (x_{e_i}, y_{e_i}, p_{e_i}, t_{e_i})\}_{i=1}^n$ captured on the near-eye event camera, estimating the gaze direction (x_d, y_d) accurately in real-time for general users.

C. System Architecture Overview

Learning to estimate gaze directly from event-based data and generalizing it to diverse users is a challenging task. To tackle this problem, our system is divided into two main functional blocks, namely the 'eye tracking block' and the 'gaze estimation block,' as shown in Fig. 2. Eye tracking block processes the event-based data stream and extracts the pupil features from the events as ellipses. The methods used in this block are image processing techniques and non-parametric statistical data analysis. The fitted ellipses representing pupils are then passed to the second block. Gaze estimation block learns to map the pupil ellipses to the gaze directions with a RNN architecture and custom angle loss function. These two blocks are detailed in the following two sections.

D. Benefits of Using Event Camera for Near-Eye Gaze Estimation

Compared to frame-based RGB camera, there are several benefits of using event camera [46] for gaze estimation on XR headsets.

Low Latency: Eye tracking and gaze estimation is one of the starting points of the XR system data flow [47], and is followed by reprojection, hologram and other functional blocks. Delay on this block propagates down to other blocks, and eventually affects the XR experience. The target latency of eye tracking for

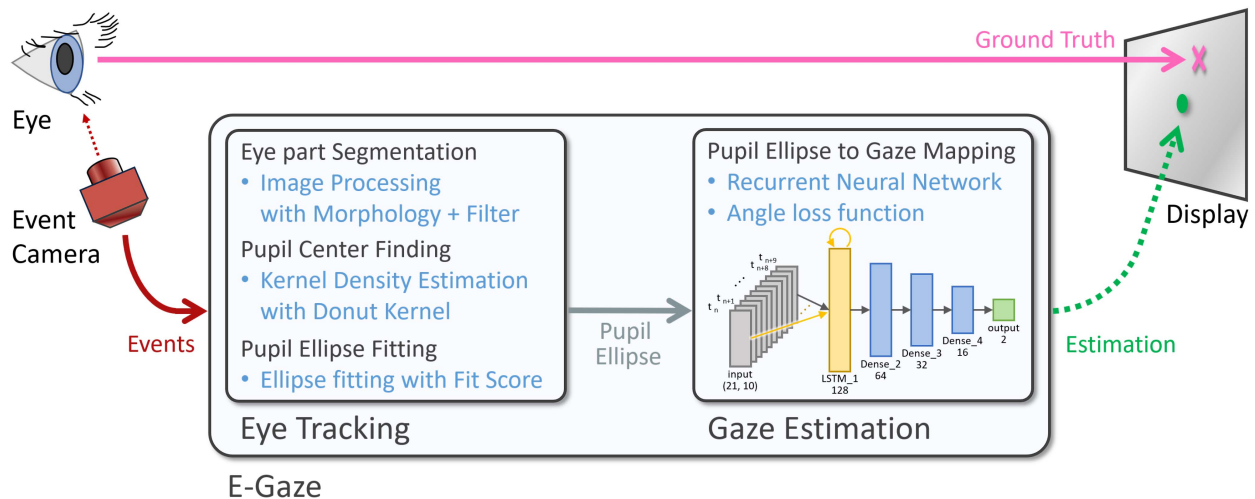


Fig. 2. Overview of the E-Gaze system architecture. Input are events representing eye movements captured by a near-eye event camera. The Eye tracking block extracts pupil features from these events in the form of ellipses. Sequences of ellipses are subsequently forwarded to the gaze estimation block, which utilizes a recurrent neural network for inference to estimate the display coordinates that represent the point of gaze.

foveated rendering is 5 ms [31]. The event camera outputs events for each individual pixel immediately when threshold is crossed and is not limited by the concept of frame: the accumulation of all pixel values. The sub-millisecond latency performance of an event camera is suitable for the low latency target.

High Temporal Resolution: The eye is the fastest moving organ in the human body, with saccadic motion reaching speeds of 700°/s. Operating at low temporal resolution, frame-based systems are less ideal for tracking the eye, as they lose movement information and suffer from motion blur. In contrast, the event camera is designed to sense individual pixel intensity differences and output events at a high frequency. With a temporal resolution of 1 μ s, the event camera can accurately capture fast and subtle eye movements. This high temporal resolution allows for the most up-to-date gaze direction to be reflected accurately on the display, enhancing the XR experience.

Low Power: The XR headset is a mobile device with limited power resources, imposing a power consumption target of 100mW for the gaze estimation functional block [31]. From a sensor's perspective, the event camera utilizes power to sense only changes generated by motion while ignoring redundant information. From an algorithm's perspective, this property results in sparse output event data that requires less power for transmission and processing. The asynchronous nature of the data stream allows power usage to be motion-dependent rather than evenly distributed over time. Power can be conserved during periods of no eye movement and allocated for critical fast motions, such as saccades and blinks.

High Dynamic Range: Environmental lighting conditions can vary significantly depending on the time and location. Frame-based cameras are constrained by global shutters and have a limited dynamic range, making them less versatile in handling diverse headset usage scenarios. In contrast, event cameras operate at the pixel level and has a much higher dynamic range. As a result, XR headsets equipped with event cameras can capture eye movements effectively across a wide range of lighting conditions, from bright sunlight to dim environments.

IV. EYE TRACKING

In this section, we describe our approach to performing eye tracking on an event-based data stream. Our methodology is developed based on a dataset [30] collected in a controlled laboratory setup, with further details provided in Section VI-A. First, we explain the accumulation of events as a set and provide a detailed analysis of event distributions within the eye parts under different movements and variations, as shown in Fig. 3. Then, leveraging the insights gained from these observations, we elaborate on the method used to extract pupil features in the form of ellipses from the event sets.

A. Accumulating Events as Sets

A single event, denoted as $e = (x_e, y_e, p_e, t_e)$, represents only a single pixel and is insufficient to form an eye feature. Our system requires a set of events for feature tracking, making the accumulation of events the initial step in our operation. Two approaches can be used to accumulate events into sets: a fixed time interval and a fixed number of events. In this context, we employ the latter approach. Since events are driven by motion, within the dataset [30] under constant illumination, a set that accumulates a fixed number of events represents a consistent amount of motion. This ensures that the event distribution representing each eye part remains stable across different sets, making the same feature extraction pipeline applicable to all sets. Furthermore, this approach preserves the asynchronous nature of events by generating sets asynchronously, which is crucial for achieving a low-latency and low-power system.

We accumulate 2000 sequential events as a set for processing since fewer events are insufficient to form eye parts, while more events can lead to overlapping eye parts. These events are accumulated to images based on their coordinates, where the value at each coordinate on the image represents the event count at that location. In the dataset [30], event polarity encodes the direction of motion, and we leverage this information as an additional dimension. Therefore, we generate three images from

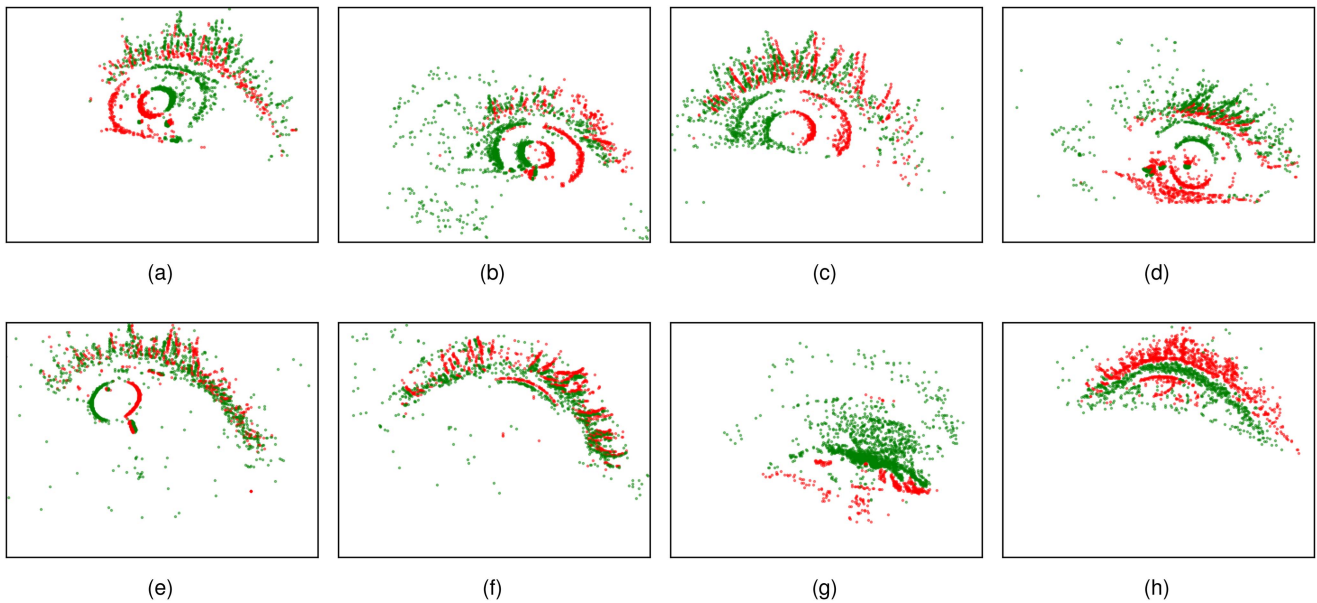


Fig. 3. Eyes captured under the event camera. Each eye's representation consists of 2000 consecutive events. (a), (b), (c), and (d) show variations in eye size, shape, and location within the event camera FoV. Compared to a frame-based camera that captures all eye parts, (e) and (f) demonstrate the motion-triggered characteristic of the event camera. In (e), an eye with a missing iris is shown, and in (f), both the pupil and iris are missing. (g) captures a full blink eye movement, and (h) is a half-blink movement.

TABLE I
EYE PARTS ON EVENT CAMERA

	Overall Density	Polar Density	Polar Cluster Distance
Eyelashes	Low	Low	Low
Eyelids & Glint	High	Very High	Very Low
Pupil & Iris	Medium	High	High

a single set of events: the first image contains only events with positive polarity, the second image only holds negative polarity events, and the third image includes all events regardless of polarity. These three images effectively encode both polar and spatio-temporal information from the set. A higher number of events accumulating at a specific coordinate indicates a relatively higher temporal density at that pixel location. A greater number of events clustered around a single pixel coordinate results in higher spatial density. All following observations and operations are conducted on these three images generated from a set of 2000 events.

B. Eye Captured on Event Camera

Eye Parts: In the event-based data stream, different eye parts exhibit unique polar and spatio-temporal characteristics, as captured in Table I. The ability to observe these differences is crucial for segmenting each eye part during feature extraction. We analyze three aspects of eye parts within a single set of events: the overall distribution of events, disregarding polarities; the distribution of events within each polarity; and the spatial distance between event clusters in opposite polarities. As shown in Fig. 4, on the event camera, eyelashes as a whole present as a cloudy mixture of positive and negative events with low

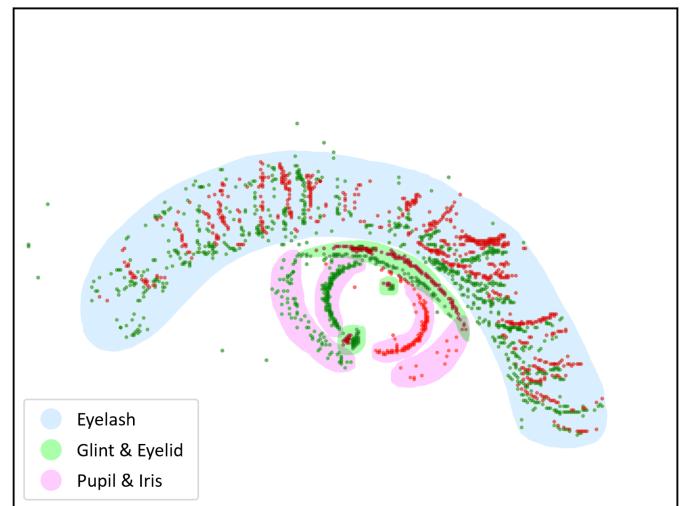


Fig. 4. Eye captured by event camera segmented in to 3 categories. Green and red dots are positive and negative polarity events, respectively. The blue area indicates regions where events are loosely distributed, spanning a large area, which corresponds to the eyelashes. Densely distributed events with positive and negative event clusters close together are covered in green, representing the glints and eyelid. Densely distributed events with positive and negative event clusters far apart are covered in pink, representing the pupil and iris.

spatio-temporal density. Eyelashes represented by only single polarity events have an even lower density, making the events nearly indistinguishable from noise. Conversely, glints and eyelids, being visually small and thin, exhibit high spatio-temporal density within each polarity individually. When events from both polarities are combined, glints and eyelids collectively have the highest overall event density among all eye parts. Notably, the two high-density event clusters in opposite polarities are adjacent to each other in space. The pupil is the target feature

for extraction from the event data stream, and along with the iris, they both display high spatio-temporal density when examining events within each polarity separately. Unlike glints and eyelids, the positive and negative event clusters are not adjacent to each other, serving as a key distinguishing factor.

Eye Movements: Under different eye movements, the polarity and spatio-temporal distribution of each eye part varies, as depicted in Fig. 3. Not all of the eye parts are present in the event-based data stream at all times, which is fundamentally distinct from frame-based data. Since the event camera operates based on motion, if a particular eye part remains motionless, it will not be represented in the event data stream. For instance, during fixation, the pupil and iris remain still, while eyelashes and eyelids are free to move. Consequently, the pupil and iris are absent in the event-based data stream, while eyelashes and eyelids trigger events and are visible. While blinking, eyelid closes and eyelashes brush down and up. This motion creates two loose event clusters representing eye lashes in opposite polarity, and pupil and iris are not clearly visible. There are also involuntary eye movements such as spontaneous blink and ocular microtremors. It is challenging to distinguish eye parts under multiple movement with one or more parts absent.

C. Eye Parts Segmentation With Image Processing

Observations and image processing techniques play a critical role in the segmentation of eye parts in event-based data. Each eye part is segmented using object masks, with each mask spatially covering its respective eye part. As illustrated in Fig. 4, there are three total masks: the eyelid and glint mask, the eyelash mask, and the pupil and iris mask. The event data stream is first preprocessed to identify and filter out noise events. Subsequently, the masks are created with morphological operations based on insights gained from the analysis of event distribution in the eye.

Noise: The event-based data stream inherently contains noise. Events generated at random times and coordinates, not triggered by motion, are considered noise. As these events are randomly generated, they do not exhibit high spatio-temporal density. To effectively filter out this noise, a box filter is separately applied to the positive and negative images to calculate the sum of events near specific coordinates. A high sum at a coordinate indicates the presence of movements and strong features in the area. Coordinates with a sum below a specified threshold on either the positive images, negative images, or both images are flagged as potential noise events. These noise events are removed for certain stages of the processing pipeline.

Eyelid and Glint: Eyelids and glints share similar distribution characteristics and are targeted together. When there are movements in glints or eyelids, two high spatio-temporal density clusters in opposite polarities appear adjacent to each other. To create the mask for glints and eyelids, noise events are initially removed from the data. Dilation is then applied separately to the positive and negative images to enhance the high-density features. Given that the positive and negative clusters are adjacent, the overlapping areas of the dilated clusters represent the locations of glints and eyelids. Thus, an intersect operation

is performed between the two images to identify these areas accurately. To ensure that all events associated with glints and eyelids are included within the mask, the overlapping regions undergo an additional round of dilation to expand coverage. The results are shown in Fig. 5(a) and (e). There is no need to further separate eyelids from glints, as the final target eye feature is the pupil, and all other components are removed.

Eyelash: The image containing all events is morphed and combined with the eyelid mask to create the eyelash mask. Events representing eyelashes are distributed loosely, even with events in both polarities present in the same image. Therefore, individual polarity images are unsuitable for this purpose, and noise events are not masked out since they could weaken the feature of eyelashes. Two intermediate masks are required: the blur mask and the morph mask. The blur mask is generated by applying a median filter and serves to eliminate unwanted events while enhancing eye part features. The morph mask results from two rounds of closing followed by one round of opening morphological operations. The shape of eyelashes is a wide horizontal cluster of numerous short and thin vertical strokes. Therefore, to connect all these small strokes into one large chunk representing the eyelashes, the first closing operation for the morph mask utilizes a horizontal rectangular kernel. Two subsequent closing morphological operations are then applied using a disk kernel. Finally, to obtain the eyelash mask, the union of the eyelid mask and the morph mask is intersected with the blur mask. Since the current mask may cover the pupil and iris, only the largest blob is kept to form the eyelash mask, as shown in Fig. 5(b) and (f).

Pupil and Iris: Since the pupil and iris are the only two remaining eye parts without masks, we create the pupil and iris mask by inverting the mask that covers all the other eye parts. The noise mask, eyelid and glint mask and eyelash mask are unified and then inverted to create the pupil and iris mask. Only the events that are covered in the pupil and iris mask, as illustrated in Fig. 5(c) and (g), are retained for the next processing stage. Events representing the pupil and iris have similar distribution characteristics spatially, temporally, and polarly. Therefore, image processing operations alone are insufficient to extract the pupil. The approach to separate the pupil from the iris is discussed in the following subsection.

D. Pupil Center Finding With Kernel Density Estimation

After masking out noise and most of the eye parts, the remaining events are distributed within two concentric circles, with the inner circle representing the pupil and the outer one corresponding to the iris. The segmentation of the iris and pupil involves two main steps: locating the center of the concentric circles and then separating the events based on their distance from the center. To locate the center of the circles, a non-parametric statistical method known as two-dimensional kernel density estimation (KDE) is applied. In this method, 2D kernels are placed on each event coordinate, and the sum of all these kernels forms the kernel density estimate. To optimize latency, we use the FFT (fast Fourier transform) variant of KDE [48], known for

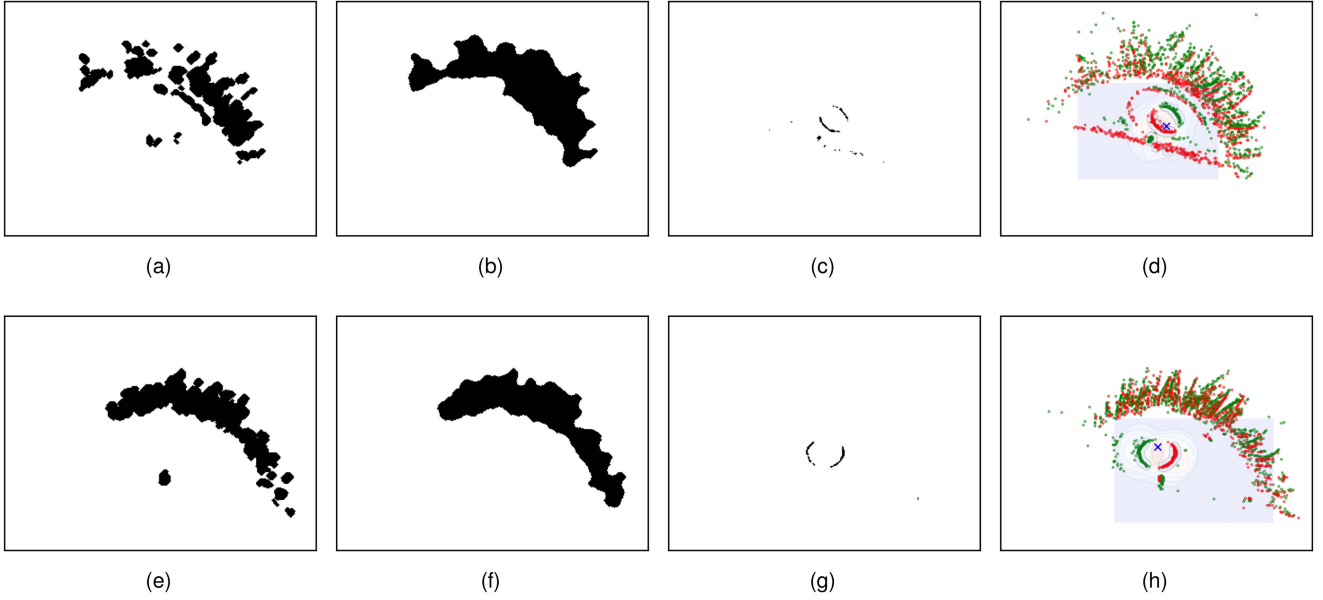


Fig. 5. Intermediate eye tracking results of two event sets. Each row shows the intermediate results of the same event set, and each column shows the results of two event sets processed by the same intermediate stage. (a) and (e): Eyelid and glint mask, which is the overlap of events in two polarities. (b) and (f): Eyelash mask generated with sparsely distributed events. (c) and (g): Pupil mask created by eliminating eyelash, eyelid, glint, and noise. (d) and (h): Pupil inner region is located by the maximum response (blue cross) of the kernel density estimation with a donut kernel.

its speed advantages [49]. Additionally, a donut-shaped kernel is introduced to enhance the robustness of center detection.

Taking inspiration from the circle Hough Transform, we employ a 2D 2-norm box kernel, which is a cylinder in shape, to amplify the center location of the pupil. The bandwidth of this kernel determines the radius of the cylinder and is intentionally set larger than the pupil's radius. The center of the pupil is where all these kernels overlap, creating a peak value. However, due to variations in eye movement, this maximum value can sometimes fall between the pupil and iris. To address this, we apply a 2D 2-norm box KDE with a smaller bandwidth to mask out the regions corresponding to the pupil, iris, and the surrounding area. This result is essentially equivalent to employing a KDE with a cylindrical shell kernel, i.e., the donut kernel. To conserve computational resources, we reuse the pupil and iris mask to exclude their respective areas, rather than employing a KDE with a smaller bandwidth. Having identified the coordinates within the pupils, as shown in Fig. 5(d) and (h), we can now isolate the events located at a specific distance, representing the pupil.

E. Pupil Ellipse Fitting With Fit Score

To acquire the ellipse describing the pupil, we apply elliptical least square fitting [50] on the event coordinates. This ellipse is defined by its center coordinates, height, width, and rotation, represented as $Ellipse_{pupil} = (x_p, y_p, h_p, w_p, \phi_p)$. Despite multiple stages to isolate pupil events from other events, events that do not belong to the pupil can still be included in the data used for fitting. Hence, we introduce a fit score to evaluate the quality of each fit after fitting. The score is calculated by initially applying an affine transformation to the 2D vector space, which transforms the fitted ellipse into a unit circle, ensuring consistent

scoring across different ellipse fits. The fit score is determined by subtracting one from the mean distance between the events representing the pupil $\mathcal{E}_{pupil}$ and the edge of the unit circle:

$$Fit_Score(Ellipse_{pupil}, \mathcal{E}_{pupil}) \triangleq 1 - \frac{1}{n} \sum_{i=1}^n \left\| R(\phi_p) \left(\begin{bmatrix} x_{e,i} \\ y_{e,i} \end{bmatrix} - \begin{bmatrix} x_p \\ y_p \end{bmatrix} \right) \oslash \begin{bmatrix} w_p \\ h_p \end{bmatrix} \right\| - 1$$

An ellipse with a fit score below a specified threshold is regarded as a poor fit and is therefore discarded, and the next set of events are processed to fit a new ellipse. Conversely, an ellipse with a good fit score proceeds to the next stage for gaze estimation.

F. Region of Interest Latency Optimization

To optimize system latency, the eye tracking process skips the morphology and kernel density estimation steps if the pupil features were successfully extracted from the previous event set. The pupil's center coordinate and dimensions in the previous event set serve as hints for the current event set, defining a region of interest (ROI) that includes the events representing the pupil. Due to the event camera's high temporal resolution and the fixed event number accumulation approach, in the dataset [30], each event set represents a consistent amount of movement, ensuring that the pupil in two consecutive event sets remains close. In contrast, with a fixed time interval approach, such as frame-based data or fixed time window event accumulation, the pupil in the following frame can be located farther away when eye movements are rapid. The ROI is defined with the same center and rotation as the previous pupil, but the width and height are expanded to account for pupil movement. All events within the

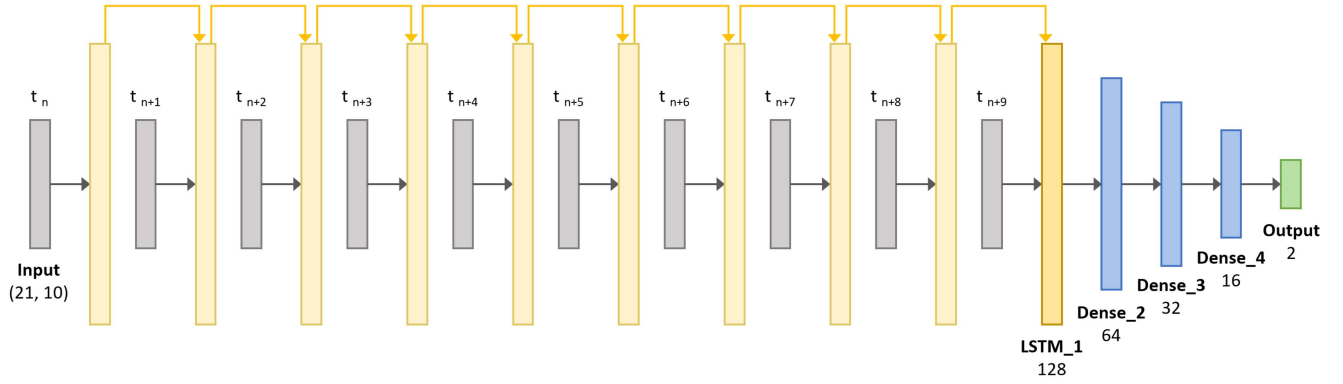


Fig. 6. Illustration of the gaze estimation architecture. The input to the network is a sequence of 10 consecutive vectors. Each vector is 21-dimensional and encodes a pupil ellipse. The LSTM layer processes the vectors in the sequence one by one. After the 10th vector is processed, the output is passed to the following three dense layers. The network's output is the estimated pixel coordinate of the gaze point on the display, (x_d, y_d) .

ROI are treated as pupil events and are processed for ellipse fitting and fit score evaluation.

V. GAZE ESTIMATION

In this section, we introduce the gaze estimation architecture, which is a combination of a RNN and fully connected layers, as illustrated in Fig. 6, and elaborate on the angular loss function.

A. Input Preprocessing for Generalization

Examined in work [51], the outer product of the vector comprising the ellipse parameters with itself encodes more information of the pupil features and demonstrates better generalization across different subjects. Initially, we augment the ellipse parameter vector by appending a '1' to preserve the 1st order combination during matrix multiplication. The parameter vector is then multiplied by its transpose, resulting in a 6×6 square matrix that captures both 1st and 2nd order combinations. However, the indices representing 2nd order combinations appear redundantly in the off-diagonal section. Retaining only the lower-triangle indices and the diagonal indices of the square matrix suffices, and discarding others reduces the architectural parameters. The triangular matrix is flattened into a vector with 21 elements, which serves as input to the neural network. All of the 21-dimensional vectors representing the extracted ellipses from the event sets are standardized to have a mean of 0 and a variance of 1 for each subject independently.

B. Gaze Estimation RNN

Contrasting with a single pupil feature extracted at a specific moment, a sequence of pupil features captures motion and encodes more information about the direction and velocity of eye movement [52]. To learn from pupil motion, which is a time series, the first layer is an LSTM layer [53], which inputs a sequence of 10 vectors. Each vector has 21 elements, and encodes a single pupil ellipse. The LSTM layer has 128 internal units, Tanh activation function, and Sigmoid recurrent activation function. Following the LSTM layer, there are three dense layers with 64, 32, and 16 units, and their activation function ReLU

is paired with the He uniform variance scaling initializer. The output layer consists of a dense layer with 2 units, assigned to estimating the coordinates (x_d, y_d) of the pixel under gaze on the display. This architecture comprises 5 layers, with each layer employing L1 kernel regularization. An illustration of the architecture is provided in Fig. 6.

C. Angular Loss Function

In the gaze estimation dataset, the label is the display pixel coordinate, which represents the intersection point of the display plane and the gaze vector. Typically, the primary optimization objective of algorithms is to minimize the distance difference between the estimated and ground truth coordinates. Nevertheless, gaze estimation results are evaluated using the mean absolute error of the angle between two vectors in space: one extending from the eye to the ground truth pixel and the other extending from the eye to the estimated pixel. Assume the center of the eye as the origin $(0, 0, 0)$ and consider the display as a plane parallel to the xy -plane at a distance z_d . We can represent the eye-to-estimation vector as u_i and the eye-to-ground truth vector as v_i in three-dimensional space:

$$u_i \triangleq [x_{e,i}, y_{e,i}, z_{d,i}]^T \in \mathbb{R}^3$$

$$v_i \triangleq [x_{g,i}, y_{g,i}, z_{d,i}]^T \in \mathbb{R}^3$$

The angle error can be expressed as:

$$\theta_{\text{error}}(u_i, v_i) \triangleq \arccos \left(\frac{u_i \cdot v_i}{\|u_i\| \|v_i\|} \right)$$

The commonly used optimization target, $d(u_i, v_i)$, involves multiple calculation steps to reach the result evaluation metric, $\theta_{\text{error}}(u_i, v_i)$. There is room for improvement by establishing a stronger correlation between these two metrics. Additionally, the labels are pixel coordinates on a flat display, not on a curved circular arc display. Because the subject's eye is not equidistant from all individual pixels on the flat display, between u_i and v_i , the identical difference in distance on the display leads to varying angular difference between the center and edges of the flat screen, as illustrated in Fig. 7. Since our focus lies in estimating the gaze angle, it is crucial to allocate more attention

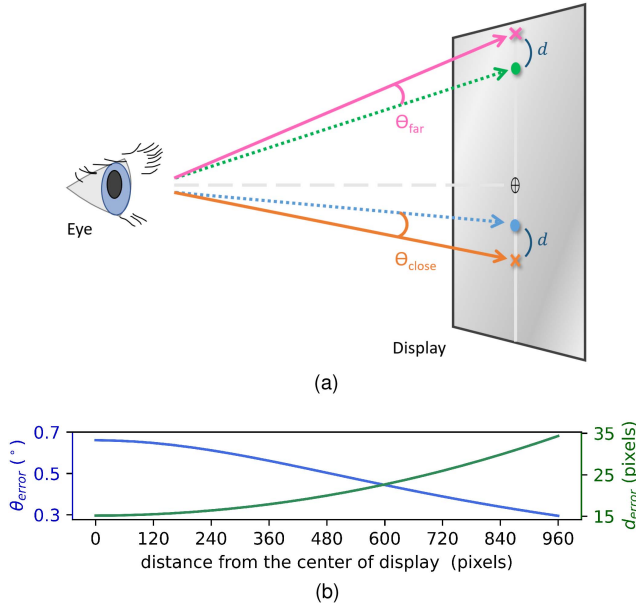


Fig. 7. (a) Two pairs of ground truth (pink cross) and estimation (green dot) are shown along with another pair of ground truth (orange cross) and estimation (blue dot). The pink-green pair is farther from the black display center than the orange-blue pair. Although both pairs have the same pixel distance error d on the display, they exhibit different angular errors ($\theta_{close} \neq \theta_{far}$), and in fact, $\theta_{close} > \theta_{far}$. This demonstrates that the same pixel distance errors do not necessarily indicate the same degree errors. (b) The relationship between varying distance errors and angular errors from the center of the display to the edge. A fixed distance error of 10 pixels results in different angle errors depending on the ground truth and the estimation pair's distance to the center of the display (blue line). On the contrary, a fixed 1° angle error results in varying distance errors (green line), with error variations of up to 20 pixels.

to regions closer to the center of the flat display, where equivalent distance errors result in larger angular differences. Attention should be distributed based on angle rather than uniformly based on distance. To address these issues, we introduce the angular loss function within the architecture to minimize the angle between two vectors, rather than the distances between two coordinates. The angular loss function is the mean square angle difference between the ground truth gaze direction and the estimated gaze direction.

$$\mathcal{L}_{angular}(u, v) \triangleq \frac{1}{n} \sum_{i=1}^n \theta_{error}(u_i, v_i)^2$$

VI. EXPERIMENTS

First we describe the dataset [30] collection method and characteristic in this section. Then, we explain the calibration process for individual subjects, and elaborate the training setup.

A. Dataset

The dataset was published by A. N. Angelopoulos et al. [30] and was collected using the DAVIS346 camera, capable of simultaneously capturing event-based and frame-based data. Both of the data formats have a resolution of 346×260 pixels, with each subject recorded for a duration of 330 seconds. As illustrated in Fig. 8, the data collection involved 24 subjects

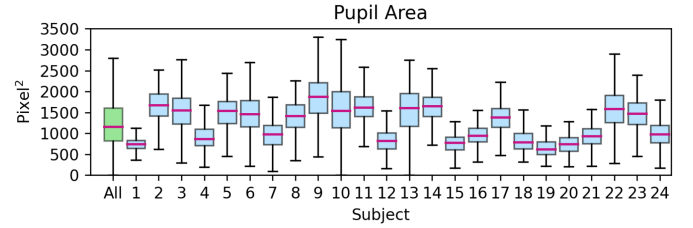


Fig. 8. This box plot shows the pupil area distribution, and demonstrates the pupil area variation across and within subjects. The blue boxes are the pupil area distributions of the 24 subjects and the green box is the pupil area distribution of all the subjects combined.

with various eye sizes, eye positions, and eye characteristics. A separate camera was positioned near each eye to ensure that only the respective eye was within the camera's field of view (FoV). Subjects rested their chin on a support station to maintain a fixed distance between their eyes and the display, with the eyes positioned in the center of the display. Eye movements were recorded as subjects tracked a stimulus displayed on the screen with their eyes. The stimulus is a green cross displayed against a black background, and the pixel coordinates of the stimulus serve as the ground truth labels. The stimulus pattern includes random saccades and smooth pursuit motions. Random saccades are positioned at random intersections of an 11×11 grid, covering the subject's $64 \times 96^\circ$ FoV. Additionally, the dataset includes movements typically considered as noise in eye tracking, including tremors, blinks, and microsaccades.

The dataset lacks labels for eye part segments and eye activities, such as blinking. However, to evaluate the effectiveness of eye tracking, a segmentation map is necessary. Thus, we utilize the EllSeg framework [34], a robust eye part segmentation method, to generate baseline segmentation and pupil ellipses by processing the frame-based data. To assess gaze estimation performance, we align our comparison setup with other state-of-the-art works. We specifically focus on random saccades within the $20 \times 40^\circ$ FoV, which includes the 16 central intersections of the grid. This is the default setup for experiments, unless stated otherwise. While the dataset includes both event-based and frame-based recordings, our proposed system exclusively utilizes the event data stream. However, the stimulus coordinate labels are associated only with frames and not events. To enable training with the event data stream, event labels are necessary. Consequently, we process an event to carry the same label as the frame captured immediately after the event, and a set of events is labeled with the same label as the latest event within the set.

B. Calibration

Eye sizes and the relative positions of eye parts vary among individuals. Additionally, even if the HMD is used by the same person, the relative position of the eye and the camera can differ each time the HMD is worn. To account for both intra- and inter-individual variations, calibration is necessary. The calibration process consists of two steps: offsetting the eye position and calibrating the model. Calibration is performed immediately after an individual finishes wearing the head-mounted display. To

improve the generalization of the eye tracking system, we offset the eye position as if it were at the center of the camera's FoV. We obtain a bounding box of the eye by applying median filters and morphological operations to the first event set, containing 4000 events. The offset is determined as the difference between the center of the bounding box and the center of the 346×260 event space. This offset is calculated once and applied to all subsequent sets. The calibration of the gaze estimation network involves a 16-point calibration process. The system records the subject's gaze while they view 16 random saccade stimuli within the $20 \times 40^\circ$ FoV and fine-tunes the pre-trained model using this new recording. The model undergoes calibration after offsetting the eye position, and this calibration is performed once for each wear.

C. Training

Consistent with prior works [26], [27], [30], [32], [35], the focus is on random saccades within a $20 \times 40^\circ$ FoV. The eye tracking block extracts pupil ellipses from the target event data stream. To increase the number of pupil ellipses in the training dataset, we adopt a method where we replace the oldest 10 events in the current set with 10 subsequent events, rather than accumulating 2000 subsequent events after each ellipse fitting. Starting with the initial pupil ellipse, we collect every 200th ellipse to create a series comprising a total of 10 pupil ellipses. This series serves as the input unit for the proposed RNN architecture. For each subject, the series is standardized and divided into training, validation, and test sets in a 70%/15%/15% ratio. We apply the leave-one-out cross-validation method for training and testing the models. Given a total of 24 subjects, we train 24 individual models, using each subject as an unseen entity during its dedicated model training iteration. For example, model 2 is trained on data from all subjects except subject 2 (i.e., subjects 1 and 3 to 24), and is subsequently tested using subject 2's data. The proposed gaze estimation model is trained with the Adam optimizer that has exponential decay with specific hyperparameters: an initial learning rate of 0.0002, a decay rate of 0.98, and a decay step of 1000. The loss function is the proposed angular loss function. We trained the network for 50 epochs with a batch size of 10. The laptop used for training and evaluating the gaze estimation system is equipped with an 8-Core Ryzen 9 5900HS CPU and an RTX 3070 laptop GPU. The system is configured with CUDA 11.5 and CuDNN 8.3, running TensorFlow version 2.6.0 and Python 3.8.

VII. RESULT

In this section, we first evaluate the eye tracking block using the metric Intersection over Union (IoU). Then, we assess the gaze estimation accuracy by comparing it to state-of-the-art near-eye gaze estimation systems. Finally, we report the achieved system latency.

A. Eye Tracking Accuracy

The eye tracking block extracts pupil ellipses as features, and its effectiveness is evaluated based on how accurately the

extracted pupil ellipse matches the actual pupil. Performance is assessed using two metrics. The first metric is the euclidean distance between the estimated pupil ellipse center and the actual pupil center. A center distance below 3pixels is considered good, and lower values are preferred. The second metric is the IoU score, also known as the Jaccard Index. IoU is a widely used segmentation metric that quantifies the similarity between two arbitrary shapes. A high IoU score indicates a high similarity and high overlap between the shapes. A higher IoU score is desirable, and typically, an IoU score above 0.5 is considered satisfactory, while a score of 1 represents a perfect match. For two shapes, denoted as A and B (representing ground truth and prediction), the IoU score is calculated as follows:

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

We have to generate a pupil ellipse that serves as the reference using an existing frame-based eye segmentation algorithm because the gaze point is the only label dataset [30] contains. Frames recorded in parallel with events are processed using the EllSeg framework [34] alongside RITnet [33], resulting in one reference pupil ellipse obtained from each frame. Pupil ellipses estimated by the eye tracking block are asynchronous, and there can be multiple estimations or no estimations between two frames. We compare only the estimations that are immediately followed by a frame, without any other estimations in between. We calculate center distances and IoU scores using the pairs of estimations and reference pupil ellipses.

Fig. 9 presents the center distance and IoU score of the pupil ellipses for all subjects, illustrating the effectiveness of the eye tracking block. Most center distances are below 3pixels, with an overall median of 1pixel, indicating accurate center estimation. IoU scores, when considering all subjects combined, are predominantly above 0.7, with a median value of 0.8, indicating a high level of accuracy. The figure also demonstrates the ability of the proposed eye tracking method to extract pupil ellipses from different subjects with diverse eye characteristics. Across all subjects, the method achieves center distances below 5pixels, a median center distance below 1.5pixels, and IoU scores mostly above 0.5, with a median IoU score above 0.75. Furthermore, the interquartile range (IQR) is small and remains consistent between subjects, demonstrating the block's stable performance both intra- and inter-subjects.

B. Angular Loss Function

To assess the effectiveness of the angular loss function, we train models using two different loss functions: the standard mean square error (MSE) and the newly proposed angular loss function. Two setups are considered: one using data from all subjects except subject under test within the $20 \times 40^\circ$ FoV, and the other using data from subject under test within the $48 \times 85^\circ$ FoV. We first focus on accuracy in degrees, as it is a common metric for evaluating gaze estimation systems. When testing with the $20 \times 40^\circ$ FoV setup, Table II demonstrates that the accuracy with calibration improves by 16.7% when using the angular loss function. To evaluate the impact on a larger FoV, we

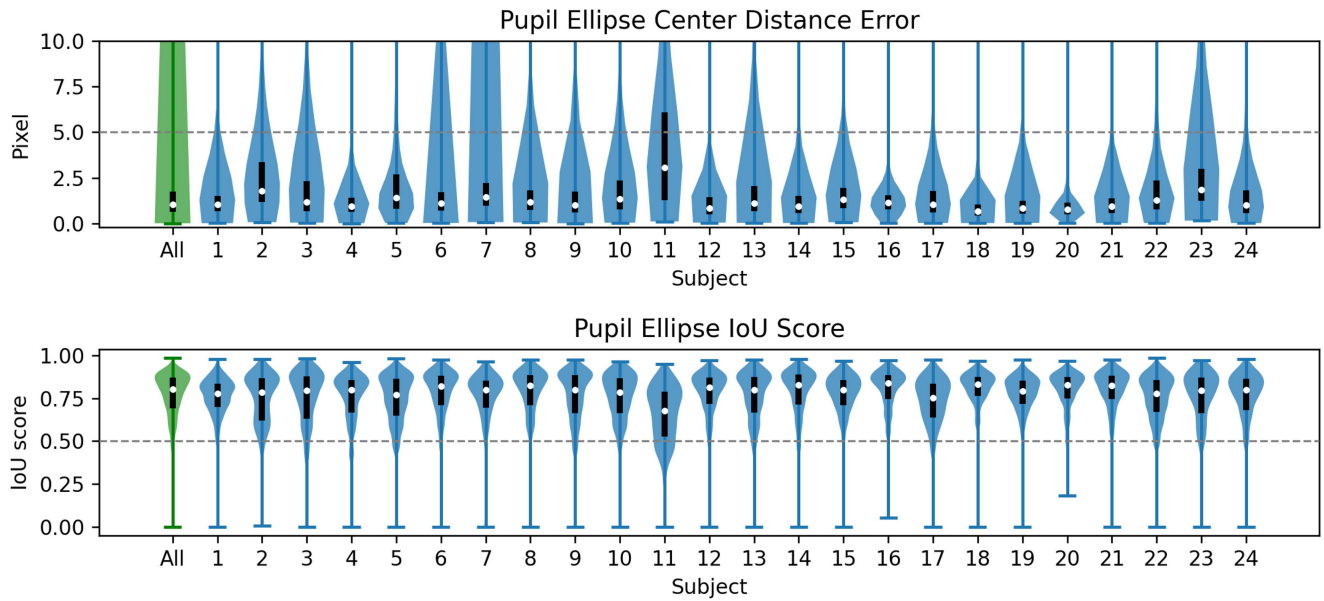


Fig. 9. This violin plots and box plots illustrating the comparison between the estimated and reference pupil ellipses. In both plots, the first column represents data from all subjects collectively, while the subsequent columns show data for each individual subject. In the upper plot, which depicts the pupil ellipse center distance error, the pixel distances of the estimation and reference pairs are mostly below the 5-pixel horizontal line, with majority of the third quartiles falling under 2.5pixels. In the lower plot, representing IoU scores, an IoU score exceeding the 0.5 horizontal line is considered good similarity. Virtually all estimations surpass this threshold, with an overall median IoU score of 0.8. This figure demonstrates the effectiveness and consistency of the eye tracking block, both intra- and inter-subjects.

TABLE II
LOSS FUNCTION COMPARISON

Metric		Gaze Estimation Accuracy($^{\circ}$)		Distance Error(pixel)	
		Angular	MSE	Angular	MSE
20 $\times$ 40 $^{\circ}$	No Calibration	4.23	4.05	85.80	83.31
	With Calibration	0.69	0.83	19.23	18.70
48 $\times$ 85 $^{\circ}$	Overall	0.68	0.70	23.34	21.54
	Within 20 $\times$ 40 $^{\circ}$	0.79	0.88	20.59	17.40
	Outside of 20 $\times$ 40 $^{\circ}$	0.67	0.67	23.66	21.97

trained a model using the complete 48 $\times$ 85 $^{\circ}$ FoV dataset. The results indicate that the improvement within the 20 $\times$ 40 $^{\circ}$ FoV reaches 10% with the angular loss function, while maintaining accuracy outside of the 20 $\times$ 40 $^{\circ}$ region and enhancing the overall accuracy for the full 48 $\times$ 85 $^{\circ}$ FoV. The two rightmost columns in Table II shows the distance evaluation results for the same setup. While the MSE loss function yields better distance results than the angular loss function across all testing setups, it is evident that this dominance does not translate to the actual gaze estimation accuracy evaluation metric. The angular loss function outperforms the MSE loss function in most of the setups. As illustrated in Fig. 10(a), estimations closer to the display center have a stronger concentration around the ground truth targets. This demonstrates that the angular loss function encourages the model to minimize angular differences rather than distance. By using the angular loss function, attention of the model is directed toward a metric that correlates more closely with the evaluation metric, resulting in improved accuracy.

C. Gaze Estimation Accuracy

Gaze estimation accuracy is a measure of the difference between the ground truth coordinates and the estimated coordinates on the display. Comparisons between different works utilize the angle between the eye-to-ground truth vector and the eye-to-estimation vector as the metric. The eye motion we target is random saccades in the 20 $\times$ 40 $^{\circ}$ FoV to align with other works [26], [27], [30], [32], [35].

A total of 24 subjects are in the dataset [30], and we assess the gaze estimation block by training the model with data from 23 subjects and testing it on the 1 remaining unseen subject. There are 24 such model with its corresponding subject as the unseen subject. We first evaluate the 24 uncalibrated models using the corresponding unseen subjects. In this setup, our system can estimate gaze with an average accuracy of 5.83 $^{\circ}$ on the unseen subject, with a median accuracy of 2.47 $^{\circ}$, as shown in Fig. 11(a). First, without calibration, we evaluate the 24 models with the corresponding unseen subject. In this setup, our system can estimate gaze with an average of 5.83 $^{\circ}$ accuracy on unseen subject with median 2.47 $^{\circ}$ as shown in Fig. 11(a). Then, following the approach of prior works, each model is individually calibrated and tested with the corresponding unseen subject. Fig. 11(b) illustrates the estimation results for subject 22. Across all subjects, our system achieves an average gaze estimation accuracy of 0.83 $^{\circ}$ with a median of 0.55 $^{\circ}$. As shown in Fig. 11(b), the highest average accuracy for a single subject is 0.46 $^{\circ}$, with a median accuracy of 0.28 $^{\circ}$.

Table III presents a comparison of gaze estimation accuracy and system frequency between our proposed system and state-of-the-art works. Note that the reported hardware platforms and

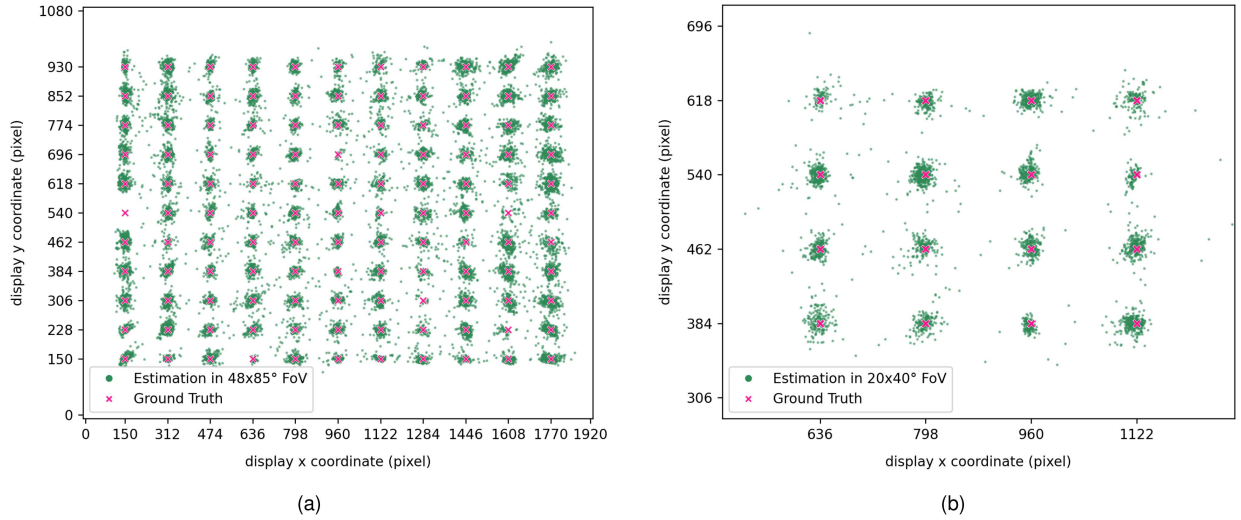


Fig. 10. Estimation and ground truth gaze points of a single subject on the display. (a) shows the gaze estimation results for the full $48 \times 85^\circ$ FoV. The average accuracy achieved in this broader FoV is 0.69° . (b) is a closer look with a FoV of $20 \times 40^\circ$, the common comparison FoV in gaze estimation studies. In this narrower FoV, the accuracy is 0.79° .

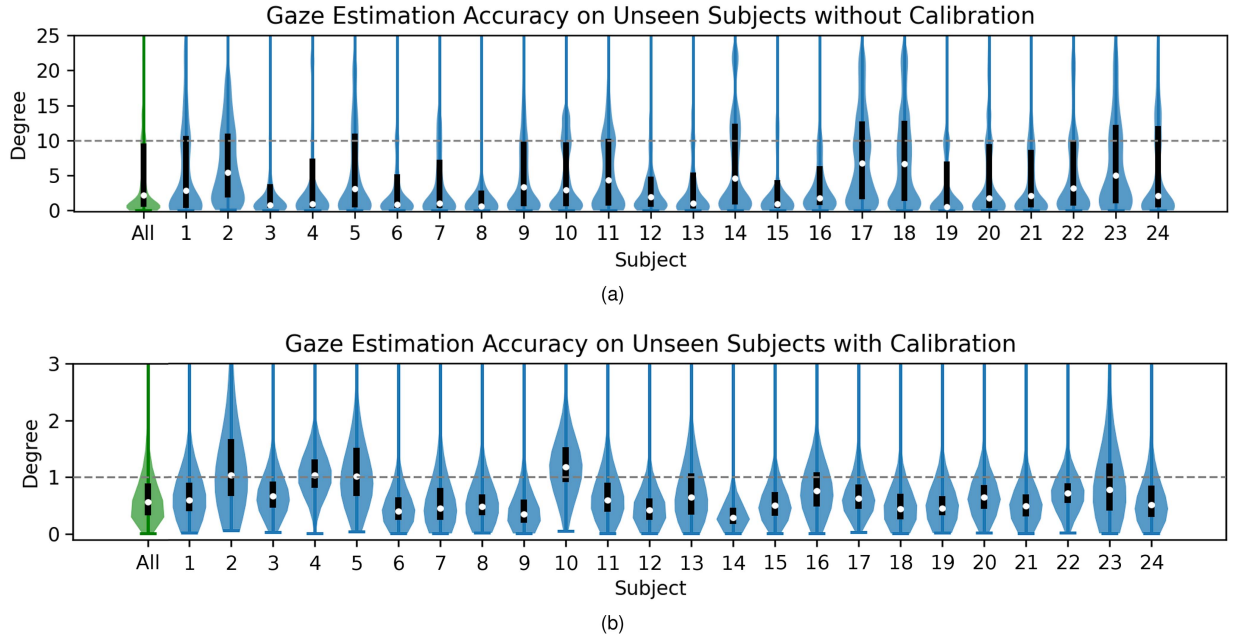


Fig. 11. Violin plot and box plot of the gaze estimation error without and with calibration. There are a total of 24 models, with each model trained without utilizing data from the target test subject. The upper plot shows the gaze estimation error on unseen subjects without calibration. The first column is the aggregate results for all subjects, revealing a median error of 2.47° . Most subjects in the remaining columns achieve a median error of under 5° . In the lower plot, the gaze estimation error for unseen subjects with calibration is shown. The overall median error is reduced to 0.55° , with a 3rd quartile at 0.89° . Across the 24 results for each subject, their 3rd quartiles mostly fall below 1.5° , with a sub- 1° median.

TABLE III
NEAR-EYE GAZE ESTIMATION COMPARISON

	Data Format		Gaze Est. Accuracy($^\circ$)	System Freq.(Hz)	Cali. Point	Hardware Platform
	Event	Frame				
EyeNet [27]		•	3.00	83	9	NVIDIA 1080Ti GPU
Y. Feng et al.[35]		•	< 0.50	30	-	NVIDIA Jetson Xavier
K. Dierkes et al.[32]		•	~ 1.00	200	5	Intel i5 CPU
NVGaze [26]		•	0.50	1,000	-	NVIDIA Titan GPU
A. N. Angelopoulos et al.[30]	★	•	0.45	10,000*	8	Theoretical Number
Ours	★		0.46	950	16	NVIDIA 3070 GPU

* This is the reported calculated pupil ellipse fitting update rate

the number of calibration points vary across these systems. In the research field of near-eye gaze estimation, the majority of works are frame-based systems, with [30] being the sole work that operates on both frame-based and event-based data. While the data formats in Table III differ, the underlying task remains the same, and our objective is to evaluate the performance of these systems in accomplishing the task. Notably, our event-based system achieves a gaze estimation accuracy of 0.46° , demonstrating its capability to estimate gaze as accurately as other frame-based systems.

D. System Latency

The time required to estimate gaze from an event set is referred to as the system's latency. It is essential to accumulate a collection of 10 pupil ellipses for the estimation process, and the gaze prediction corresponds to the final pupil ellipse. Therefore, latency is defined as the time interval between receiving the 10th pupil ellipse and performing the actual gaze estimation for that specific ellipse. The inclusion of ROI latency optimization introduces variability in the system's latency. When the pupil's location in the previous event set is known, events in the current set that represent the pupil can be selected without the need for morphological operations in the eye tracking pipeline. In cases where blinking or other noisy movements result in the system losing track of the pupil's position, the system must reacquire the pupil's location by executing the entire eye tracking pipeline. The system latency is 1ms when the pupil is successfully extracted from the previous event set, and it increases to 20ms when there is no hint available. Notably, only 2,000 out of 250,000 event sets necessitate the execution of full eye tracking pipeline, accounting for less than 1% of the event sets. Therefore, ROI latency optimization is applied 99% of the time, resulting in a notable improvement in the average system latency from 20ms to 1.025ms. This demonstrates the importance of ROI latency optimization and highlights the system's capability to operate with low latency.

The works in the comparison have reported their frequency results on three primary platforms, as indicated in Table III. Specifically, [32] conducted their testing on a CPU, [35] on a mobile GPU, and both [26], [27], as well as our work, used a desktop GPU for testing. It is important to note that the reported 10,000Hz update rate in [30] refers to pupil fit updates. This rate is derived by dividing the estimated number of events per second by the number of events required by their algorithm to fit a pupil. Therefore, it is not directly comparable to the system frequencies reported in the other works. We assume that our system operates on a one-set-per-time basis, transitioning to process the subsequent set immediately upon completing the previous one. Consequently, our system frequency is calculated based on the 1.025ms interval between the execution of two sets, which corresponds to the system latency, serving as the system's period. Our system has achieved a rate comparable to that of the leading frame-based system, NVgaze [26]. Notably, our proposed system demonstrates comparable, if not superior, performance to other near-eye gaze estimation systems, utilizing only event data streams.

VIII. LIMITATION

While the proposed system effectively performs gaze estimation using event-based data, it is not without limitations.

The need for calibration as noted in Section VI-B before every usage results from the inconsistencies in device positioning, which persist even when the same user uses the system. This requirement limits the ease of use of the proposed system, primarily due to the time-consuming nature of calibration and the need to repeat these steps prior to each session.

Additionally, another limitation is the necessity for the system to require the distance between the user's eye and the display in order to implement the angular loss function as described in Section V-C. Since this distance can differ across various headset designs, it is important information that the algorithm must have to perform accurate gaze estimation for each design.

It is essential to acknowledge that the algorithm's development was based on a dataset [30] acquired within a controlled laboratory environment characterized by fixed lighting conditions and a stable equipment setup as explained in Section VI-A. Consequently, the system's applicability is limited to controlled environments. Real-world scenarios introduce complexities, including fluctuating illumination, device mobility, reflections from eyewear, pupil dilation and constriction, and potential occlusions caused by eyelashes [54], all of which can potentially impact the performance of the proposed eye tracking and gaze estimation system.

IX. CONCLUSION

In this paper, we proposed E-Gaze, the first event-based gaze estimation system. The system can extract eye features directly and solely from the event camera and accurately estimate gaze with RNN architecture in real time. We have shared observations on how the size and motion of objects affect the event distribution, along with insights on the appropriate operations to leverage these characteristics. Although challenging, we have showed that it is possible to harness the polar, spatial, and temporal characteristics of the event-based data stream to detect eye parts fast and reliably. The pupil features we extract preserve the asynchronous nature of events and are used as input to our RNN model. By minimizing the angle error as the target instead of the distance error, our model has learned to map pupil motion to gaze direction. We believe that these contributions will advance the development of a better XR experience and lowers the barriers to performing other computer vision tasks with event camera.

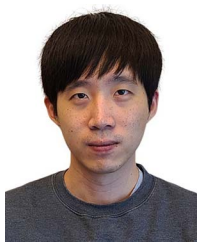
REFERENCES

- [1] M. Speicher, B. D. Hall, and M. Nebeling, "What is mixed reality?," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2019, pp. 1–15.
- [2] T. Morimoto et al., "XR (extended reality: Virtual reality, augmented reality, mixed reality) technology in spine medicine: Status quo and quo vadis," *J. Clin. Med.*, vol. 11, no. 2, 2022, Art. no. 470.
- [3] J. Gerup, C. B. Soerensen, and P. Dieckmann, "Augmented reality and mixed reality for healthcare education beyond surgery: An integrative review," *Int. J. Med. Educ.*, vol. 11, 2020, Art. no. 1.
- [4] M. J. Lee, M. Georgieva, B. Alexander, E. Craig, and J. Richter, *State of XR & Immersive Learning Outlook Report 2021*. Walnut, CA, USA: Immersive Learn. Res. Netw., 2021.

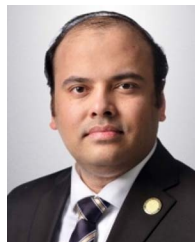
- [5] P. Fraga-Lamas, T. M. Fernández-Caramés, O. Blanco-Novoa, and M. A. Vilar-Montesinos, "A review on industrial augmented reality systems for the industry 4.0 shipyard," *IEEE Access*, vol. 6, pp. 13 358–13 375, 2018.
- [6] A. Çöltekin et al., "Extended reality in spatial sciences: A review of research challenges and future directions," *ISPRS Int. J. Geo-Inf.*, vol. 9, no. 7, 2020, Art. no. 439.
- [7] J. C. Cheng, K. Chen, and W. Chen, "State-of-the-art review on mixed reality applications in the AECO industry," *J. Construction Eng. Manage.*, vol. 146, no. 2, 2020, Art. no. 03119009.
- [8] P. Parekh, S. Patel, N. Patel, and M. Shah, "Systematic review and meta-analysis of augmented reality in medicine, retail, and games," *Vis. Comput. Ind., Biomed., Art.*, vol. 3, no. 1, pp. 1–20, 2020.
- [9] S. Rokhsaritalemi, A. Sadeghi-Niaraki, and S.-M. Choi, "A review on mixed reality: Current trends, challenges and prospects," *Appl. Sci.*, vol. 10, no. 2, 2020, Art. no. 636.
- [10] L. Hsiao, B. Krajancich, P. Levis, G. Wetzstein, and K. Winstein, "Towards retina-quality VR video streaming: 15ms could save you 80% of your bandwidth," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 52, no. 1, pp. 10–19, 2022.
- [11] J. Xiong, E.-L. Hsiang, Z. He, T. Zhan, and S.-T. Wu, "Augmented reality and virtual reality displays: Emerging technologies and future perspectives," *Light: Sci. Appl.*, vol. 10, no. 1, pp. 1–30, 2021.
- [12] G. Kramida, "Resolving the vergence-accommodation conflict in head-mounted displays," *IEEE Trans. Vis. Comput. Graph.*, vol. 22, no. 7, pp. 1912–1931, Jul. 2016.
- [13] J. Martschinke, J. Martschinke, M. Stamminger, and F. Bauer, "Gaze-dependent distortion correction for thick lenses in HMDs," in *Proc. IEEE Conf. Virtual Reality 3D User Interfaces*, 2019, pp. 1848–1851.
- [14] D. Dunn, "Required accuracy of gaze tracking for varifocal displays," in *Proc. IEEE Conf. Virtual Reality 3D User Interfaces*, 2019, pp. 1838–1842.
- [15] N. Sidorakis, G. A. Koulouris, and K. Mania, "Binocular eye-tracking for the control of a 3D immersive multimedia user interface," in *Proc. IEEE 15th workshop Everyday Virtual Reality*, 2015, pp. 15–18.
- [16] M. Kim, J. Lee, C. Jeon, and J. Kim, "A study on interaction of gaze pointer-based user interface in mobile virtual reality environment," *Symmetry*, vol. 9, no. 9, 2017, Art. no. 189.
- [17] X. Wang, J. Zhang, H. Zhang, S. Zhao, and H. Liu, "Vision-based gaze estimation: A review," *IEEE Trans. Cogn. Devel. Syst.*, vol. 14, no. 2, pp. 316–332, Jun. 2022.
- [18] B. T. Carter and S. G. Luke, "Best practices in eye tracking research," *Int. J. Psychophysiol.*, vol. 155, pp. 49–62, 2020.
- [19] A. F. Klaib, N. O. Alsrhein, W. Y. Melhem, H. O. Bashtawi, and A. A. Magableh, "Eye tracking algorithms, techniques, tools, and applications with an emphasis on machine learning and Internet of Things technologies," *Expert Syst. Appl.*, vol. 166, 2021, Art. no. 114037.
- [20] D. W. Hansen and Q. Ji, "In the eye of the beholder: A survey of models for eyes and gaze," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 3, pp. 478–500, Mar. 2010.
- [21] Y. Cheng, H. Wang, Y. Bao, and F. Lu, "Appearance-based gaze estimation with deep learning: A review and benchmark," 2021, *arXiv:2104.12668*.
- [22] A. Kar and P. Corcoran, "A review and analysis of eye-gaze estimation systems, algorithms and performance evaluation methods in consumer platforms," *IEEE Access*, vol. 5, pp. 16 495–16 519, 2017.
- [23] A. L. Yarbus, *Eye Movements and Vision*. Berlin, Germany: Springer, 2013.
- [24] J. Liu, J. Chi, H. Yang, and X. Yin, "In the eye of the beholder: A survey of gaze tracking techniques," *Pattern Recognit.*, vol. 132, 2022, Art. no. 108944.
- [25] D. Cazzato, M. Leo, C. Distanti, and H. Voos, "When i look into your eyes: A survey on computer vision contributions for human gaze estimation and tracking," *Sensors*, vol. 20, no. 13, 2020, Art. no. 3739.
- [26] J. Kim et al., "Nvgaze: An anatomically-informed dataset for low-latency, near-eye gaze estimation," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2019, pp. 1–12.
- [27] Z. Wu, S. Rajendran, T. van As, J. Zimmermann, V. Badrinarayanan, and A. Rabinovich, "EyeNet: A multi-task network for off-axis eye gaze estimation and user understanding," 2019, *arXiv: 1908.09060*.
- [28] T. Stoffregen, H. Daraei, C. Robinson, and A. Fix, "Event-based kilohertz eye tracking using coded differential lighting," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2022, pp. 2515–2523.
- [29] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128×128 120 dB 15μs latency asynchronous temporal contrast vision sensor," *IEEE J. Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, Feb. 2008.
- [30] A. N. Angelopoulos, J. N. Martel, A. P. Kohli, J. Conradt, and G. Wetzstein, "Event based, near eye gaze tracking beyond 10,000 Hz," 2020, *arXiv: 2004.03577*.
- [31] I. Hong, K. Bong, and H.-J. Yoo, "Challenges of eye tracking systems for mobile XR glasses," in *Applications of Digital Image Processing XLI*, vol. 10752. Bellingham, WA, USA: SPIE, 2018, pp. 391–397.
- [32] K. Dierkes, M. Kassner, and A. Bulling, "A fast approach to refraction-aware eye-model fitting and gaze prediction," in *Proc. 11th ACM Symp. Eye Tracking Res. Appl.*, 2019, pp. 1–9.
- [33] A. K. Chaudhary et al., "RITnet: Real-time semantic segmentation of the eye for gaze tracking," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop*, 2019, pp. 3698–3702.
- [34] R. S. Kothari, A. K. Chaudhary, R. J. Bailey, J. B. Pelz, and G. J. Diaz, "ELLSeg: An ellipse segmentation framework for robust gaze tracking," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 5, pp. 2757–2767, May 2021.
- [35] Y. Feng, N. Goulding-Hotta, A. Khan, H. Reyserhove, and Y. Zhu, "Real-time gaze tracking with event-driven eye segmentation," in *Proc. IEEE Conf. Virtual Reality 3D User Interfaces*, 2022, pp. 399–408.
- [36] W. Fuhl, T. Kübler, K. Sippel, W. Rosenstiel, and E. Kasneci, "Excuse: Robust pupil detection in real-world scenarios," in *Proc. 12th Int. Conf. Comput. Anal. Images Patterns*, Springer, 2015, pp. 39–51.
- [37] W. Fuhl, T. C. Santini, T. Kübler, and E. Kasneci, "Else: Ellipse selection for robust pupil detection in real-world environments," in *Proc. 9th Biennial ACM Symp. Eye Tracking Res. Appl.*, 2016, pp. 123–130.
- [38] W. Fuhl, G. Kasneci, and E. Kasneci, "Teyed: Over 20 million real-world eye images with pupil, eyelid, and iris 2D and 3D segmentations, 2D and 3D landmarks, 3D eyeball, gaze vector, and eye movement types," in *Proc. IEEE Int. Symp. Mixed Augmented Reality*, 2021, pp. 367–375.
- [39] C. Brandli, R. Berner, M. Yang, S.-C. Liu, and T. Delbruck, "A 240×180 130 db 3 μs latency global shutter spatiotemporal vision sensor," *IEEE J. Solid-State Circuits*, vol. 49, no. 10, pp. 2333–2341, Oct. 2014.
- [40] A. Glover and C. Bartolozzi, "Event-driven ball detection and gaze fixation in clutter," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2016, pp. 2203–2208.
- [41] C. Iaboni, H. Patel, D. Lobo, J.-W. Choi, and P. Abichandani, "Event camera based real-time detection and tracking of indoor ground robots," *IEEE Access*, vol. 9, pp. 166 588–166 602, 2021.
- [42] M. Ester et al., "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 1996, pp. 226–231.
- [43] M. Monforte, A. Arriandaga, A. Glover, and C. Bartolozzi, "Exploiting event cameras for spatio-temporal prediction of fast-changing trajectories," in *Proc. IEEE 2nd Int. Conf. Artif. Intell. Circuits Syst.*, 2020, pp. 108–112.
- [44] C. Ryan et al., "Real-time face & eye tracking and blink detection using event cameras," *Neural Netw.*, vol. 141, pp. 87–97, 2021.
- [45] T. Delbruck, R. Graca, and M. Paluch, "Feedback control of event cameras," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1324–1332.
- [46] G. Gallego et al., "Event-based vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 154–180, Jan. 2022.
- [47] M. Huzaifa et al., "Exploring extended reality with ILLIXR: A new playground for architecture research," 2020, *arXiv: 2004.04643*.
- [48] M. P. Wand and M. C. Jones, *Kernel Smoothing*. Boca Raton, FL, USA: CRC Press, 1994, pp. 182–192.
- [49] T. Odland, "Tommyod/KDEpy: Kernel density estimation in Python," Dec. 2018. [Online]. Available: <https://doi.org/10.5281/zenodo.2392268>
- [50] B. Hammel and N. Sullivan-Molina, "Bdhammel/least-squares-ellipse-fitting: V2.0.0," Mar. 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.3723294>
- [51] S. Y. Han and N. I. Cho, "User-independent gaze estimation by extracting pupil parameter and its mapping to the gaze angle," in *Proc. 25th Int. Conf. Pattern Recognit.*, 2021, pp. 1993–2000.
- [52] C. P. Cantarino, O. V. Komogortsev, and S. S. Talathi, "Benefits of temporal information for appearance-based gaze estimation," in *Proc. ACM Symp. Eye Tracking Res. Appl.*, 2020, pp. 1–5.
- [53] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [54] R. T. Al-Zubi, K. A. Darabkh, and N. Al-Zubi, "Effect of eyelid and eyelash occlusions on a practical iris recognition system: Analysis and solution," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 29, no. 08, 2015, Art. no. 1556016.



Nealsen Li (Student Member, IEEE) received the MS degree in electrical and computer engineering from Georgia Tech, Atlanta, in 2017. In 2018, he joined Qualcomm as an ASIC design engineer, focusing on the design and verification of the last level cache controller for a duration of two years. In the Fall of 2020, he joined ICSRL and is currently working toward the PhD degree with the Department of Electrical and Computer Engineering at the Georgia Institute of Technology. His research interests include VLSI Circuit and System Design, Wearables, Extended Reality, and Machine Learning. He is the recipient of the 2023 ISSCC 'Code-a-Chip' Travel Grant Award and the 2023 Georgia Tech Graduate Teaching Assistant Award.



Muya Chang (Member, IEEE) received the MS degree in computer science (CS) from the Georgia Institute of Technology, Atlanta, GA, USA and the PhD degree in electrical and computer engineering (ECE) from the Georgia Institute of Technology, Atlanta, GA, USA in 2020. He is currently a research scientist in NVIDIA, AISC, and VLSI group where he focuses on efficient deep learning accelerator design. He had been with the Integrated Circuit and System Research Laboratory (ICSRL) from 2016 to 2022, and was advised by Professor Arijit Raychowdhury. His research interests include low-power circuit/system, hardware-software co-design, machine learning.



Arijit Raychowdhury (Fellow, IEEE) received the PhD degree in electrical and computer engineering from Purdue University in 2007. He is the Steve W. Chaddick chair of the School of Electrical and Computer Engineering, Georgia Institute of Technology. He joined Georgia Tech in 2013. From 2013 to 2019, he was an associate professor and held the ON Semiconductor Junior Professorship with the department. Prior to joining Georgia Tech, he held research positions with Intel Corporation for six years and at Texas Instruments for one and a half years. He holds more than 27 U.S. and international patents and has published more than 300 articles in journals and refereed conferences. His research interests include low power digital and mixed-signal circuit design, design of power converters, signal-processors, and exploring interactions of circuits with device technologies. He is currently a distinguished lecturer of the IEEE Solid State Circuits Society (SSCS) and a mentor for IEEE Young Professionals and IEEE Women in Circuits. He serves on the Technical Program Committee of key circuits and design conferences, including ISSCC, VLSI Symposium, DAC, and CICC. He is the winner of several prestigious awards, including the SRC Technical Excellence Award in 2021, the Qualcomm Faculty Award in 2020, the IEEE/ACM Innovator under 40 Award, the NSF CISE Research Initiation Initiative Award (CRII) in 2015, the Intel Labs Technical Contribution Award in 2011, the Dimitris N. Chorafas Award for Outstanding Doctoral Research and Best Thesis in 2007, and several fellowships. He and his students have won 16 best paper awards over the years.